

* Unit -3 *

Requirement Analysis

Requirement Analysis is a process used to determine the needs & expectation of the new product. Involves frequent communication with the stake holders & end user of the product to define expectation & resolve complex & document of the key requirement.

It is a crucial Phase in Software engineering that involves gathering document & analyzing requirement for a software system. It is a foundation for the entire software development process & significantly influences the success of the project.

- (1) Stake holder identification - Identify all the stake holder including, clients, end users, developers, & tester who are involved in the system.
- (2) Gathering Requirement - Gathering requirement through interviews, survey, & workshop, studying existing document, meeting.
- (3) Modeling → DFD, ER, Finite State machine.
- (4) Specification → SRS
- (5) Review & validation.

Types of requirement :-

(1) Functional Requirement - Specify what the system should do. These are specific behaviours of the system. Often described in behaviour diagram. (Usecase diagram & Activity diagram)

(2) Non Functional Requirement - Include criteria such as performance, security, usability, scalability, reliability. These describe the quality attributes of the system.

* Requirement Analysis Process :-

- (a) Identify key Stake-holders & end users
- (b) Capture Requirement

- Hold one-on-one
- User focus groups
- Utilize use cases
- Build prototypes.

- (c) Categorize Requirement

- Functional
- Technical
- Transitional
- Operational

Principle

- (i) Focus on what not how.
- (ii) Use abstraction.
- (iii) Divide complex problem in modules.
- (iv) Keep model simple
- (v) Validate requirement

(d) Interpret & Record Requirement

- Define Requirement clearly.
- Prioritize Requirement clearly
- Carry out an impact Analysis
- Resolve conflicts
- Analyze Feasibility

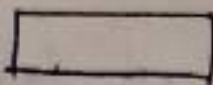
(e) Signoff

Stages of Requirement Analysis

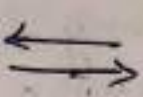
- (1) Draw the Context Diagram.
- (2) Developing a prototype (optional)
- (3) Modelling the requirement.
 - DFD = Data Flow Diagram
 - ER Diagram =
 - Data Dictionary
 - State transition
- (4) Finalize the Requirement.

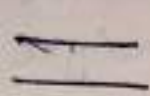
* Data Flow Diagram :-

- Level 0 (Context diagram)
- Level 1 (Overview diagram)
- Level 2 (Detailed diagram)

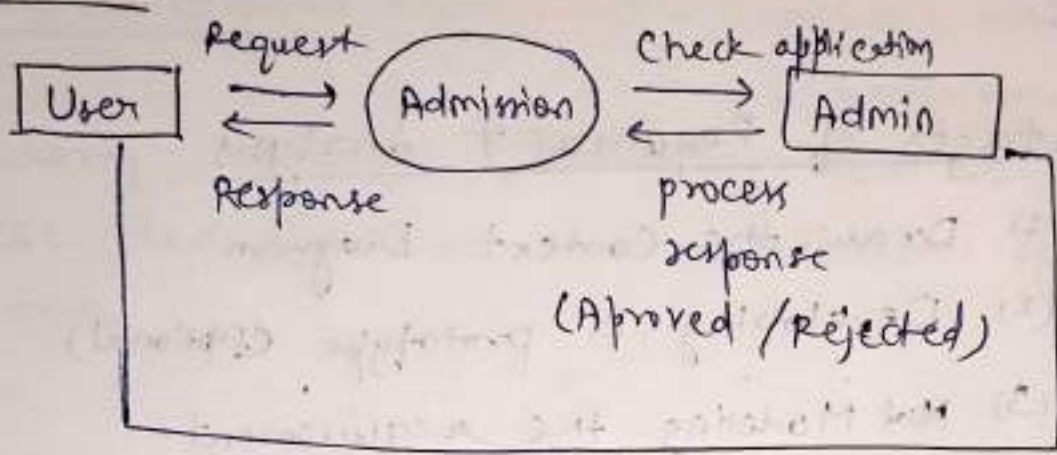
user → 

process → 

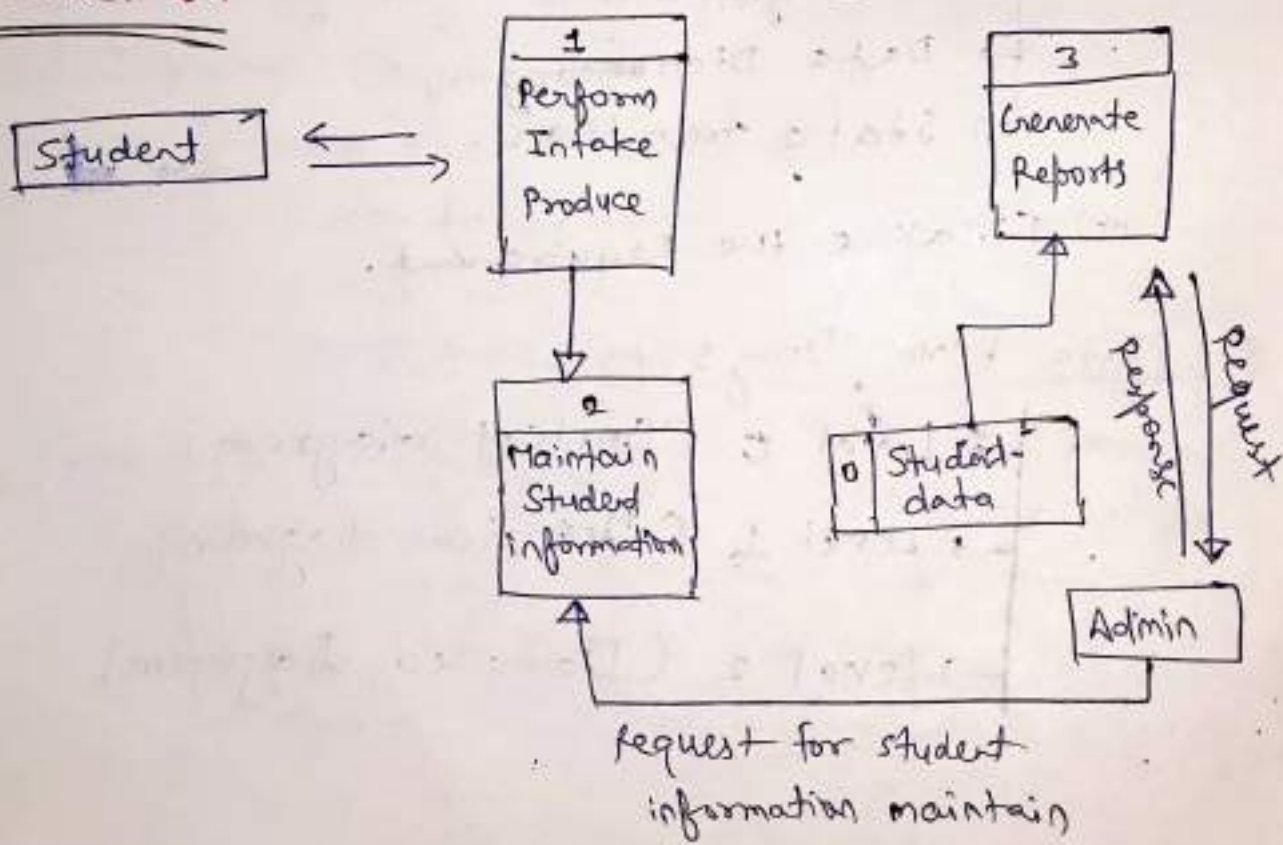
Control flow → 

Data store → 

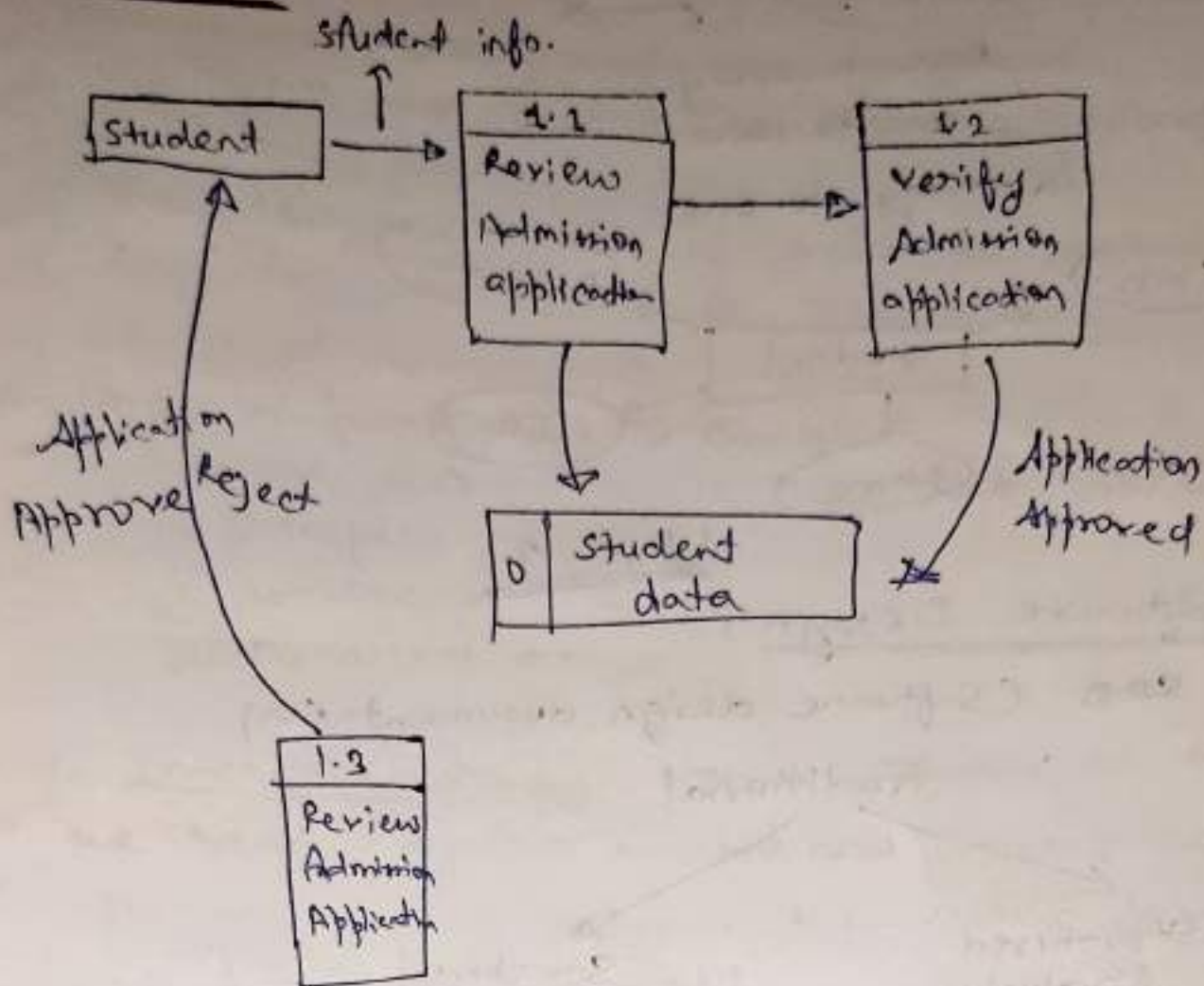
Level 0:-



Level 1:-

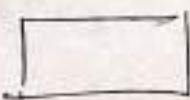


Level - 2



* Entity Relationship Diagram:-

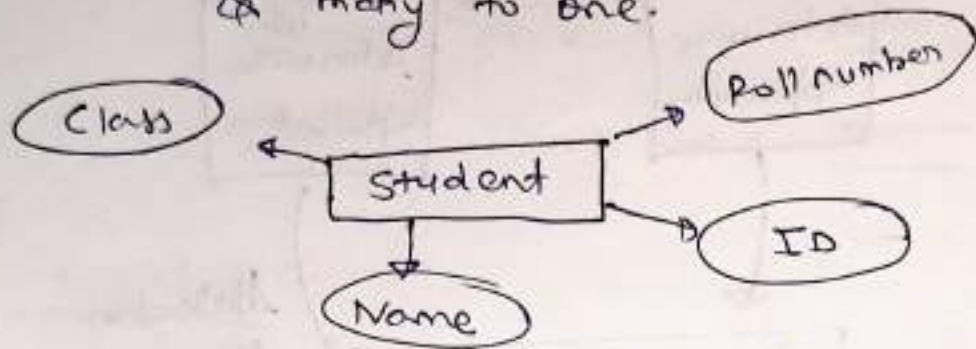
Detailed logical representation diagram relationship between data objects & entities. It's conceptual data model of an information system diagram created using ER modeling methods, or called ER diagram. Element & components → entity, Attributes & relationship.

Entity → 

Attribute → 

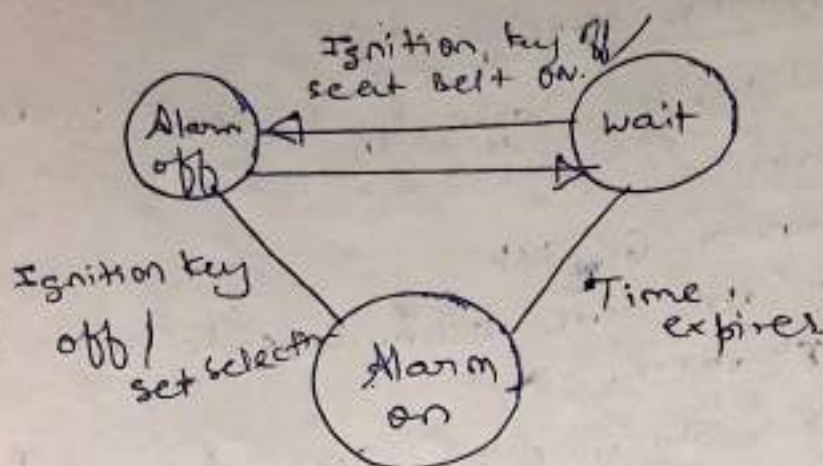
Relationship — →

one to many
one to one
& many to one.



Unit-4

Finite State Machine



State
event
Action
Transition.

*(embedded system for driver's seat belt warn)

- Reactive and event driven mode
- embedded system whose processing behaviour are dependent on state transition.
- It is represented as state event action & transition.

1.) State :- It is representation of current situation.

2.) Event :- It is representation of input to the state.

3.) Action :- It is representation of Activity to be performed by state machine.

4.) Transition :- It is movement from one state to another state.

- It is one in which number of states are finite.

* Control & process specification

Used to describe logic of process in details.
important tools-

(i) Structural English
with this we communicate with developer,
user, stakeholder.

• SRS document, SDD document.

(ii) Decision table.

(iii) Decision trees.

(iv) Pseudo Code. → In this, a rough basic
idea of functions, we get. It
includes all modules and functionality
of project.

Software Prototyping And Specification

* Software Requirements :-

- The requirements for a system is defined as what the system should do, the service that its operation. These requirements refers needs of the customer in a system.

* Types of requirements :-

- 1 :- Functional Requirements :-
 - There are statements of services the system should provide how the system should react to particular input and how the system should behave in particular situation.

- 2 :- Non-functional Requirements :-
 - There are constraints on the services or functions offered by the system.
 - They include timing constraints, constraints on the development process and constraints.

* Assignment-3 *

Q.1 Explain the Purpose of DFD's and Control flow Diagrams in structured analysis. How do they aid in understanding information & control flow?

Ans

Data Flow Diagram (DFD)

A Data Flow diagram shows how data moves within a system.

It focuses on -

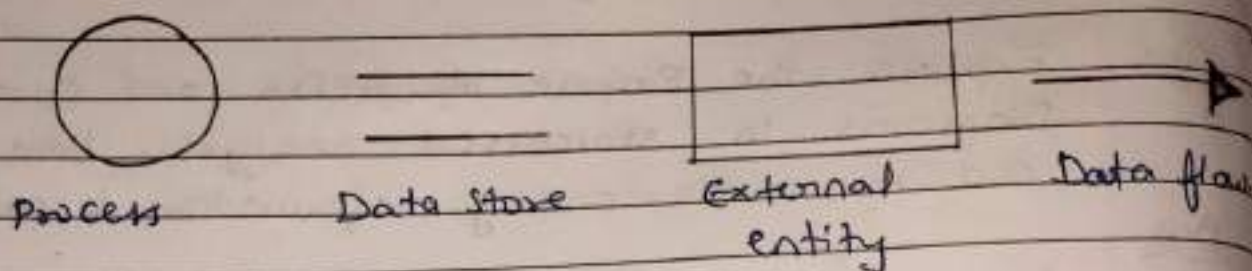
- Processes (transform table/data)
- Data stores (store data).
- External entities (input/output sources.)
- Data flows (movement of data)

Purpose of DFDs

- To visualize information flow in the system.
- To break complex systems into simple smaller processes.
- To remove ambiguity by showing what functions do with data.
- To communicate system requirements clearly to developers & users.

How DFD's help in understanding information flow

- Shows where data comes from, where it moves, and how it is stored.
- Provides a clear blueprint of data processing.
- Helps identify inefficiencies, redundancy & missing processes.



Control Flow Diagram (CFD)

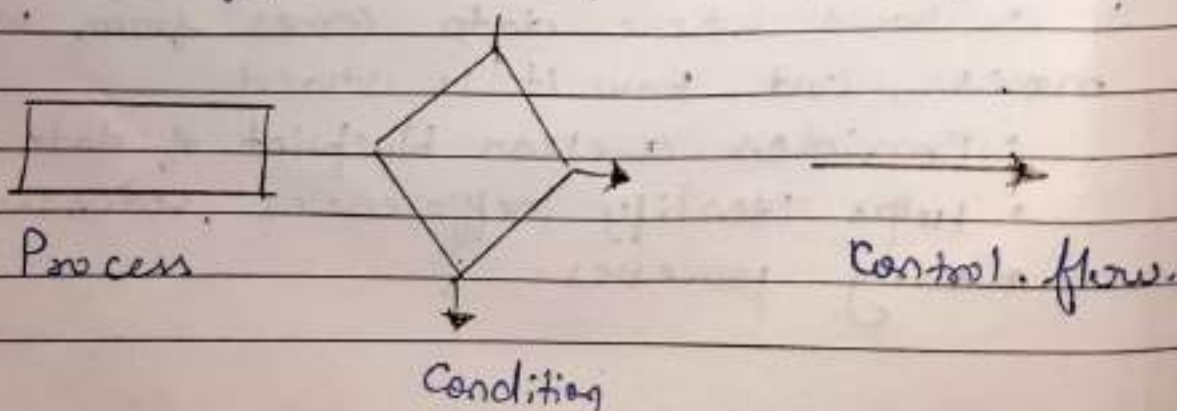
A control flow diagram shows the flow of control, i.e. order of operations, decisions, conditions & events.

Purpose of Control Flow diagram -

- To show how processes are controlled.
- To represent decisions, loops, conditions & triggers.
- Useful for real-time & embedded systems where control logic is important.

How CFDs help in understanding control flow

- Shows sequence of operation.
- Helps developers understand event-driven behaviour.
- Identify decision points & alternatives paths.



Q2 What do you understand by data dictionary?
Why accuracy is important in data dictionary?

Ans Data dictionary -

A data dictionary is a centralized repository containing details about all data elements used in the system.

It includes -

- Data name
- Data type
- Data size
- Allowed values
- Description
- Relationship

Why accuracy is important

- Prevent inconsistencies in data usage.
- Ensures correct understanding of data among developers, designers & testers.
- Helps avoid errors in database design.
- Improves communication across teams by providing a single source of truth.
- Accurate data dictionary ensures correct DFDs, ER diagrams & program logic.

If the data dictionary is inaccurate → the system will have errors, wrong outputs & integration failures.

Modularity dividing software into independent modules.

- Each module handles a part of system functionality.

Relationship - Abstraction reduces complexity; modularity organizes system into manageable components.

Both help in effective software design.

(2) Pattern :- Design patterns are general reusable solutions to common software design problems.

Ex → Singleton → Observer
→ Factory → MVC

Benefits

- Proven solutions reduce errors.
- Promote reuse
- Improve communication among developers

(3) Functional Independence

A functionally independent module :-

- Has High cohesion (does one job well)
- Has low coupling (minimum dependency on others)

Benefits

- Easier to modify
- Easier to test
- Increases reliability
- Improves code reusability

Q6 What is meant by network diagram. What are the different ways to drawing network diagram.

Ans

Network Diagram :-

A network diagram is a graphical representation of project activities and their dependencies. It shows the sequence of tasks, flows of work and critical path in project management (PERT/CPM)

It helps identify -

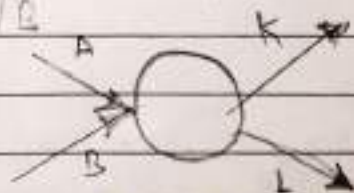
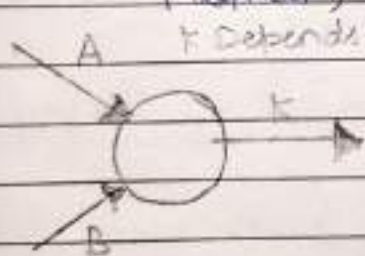
- The order of activities
- Activities that can run in parallel
- Total project time
- Critical activities (Cannot be delayed)

* Different ways to drawing Network diagram

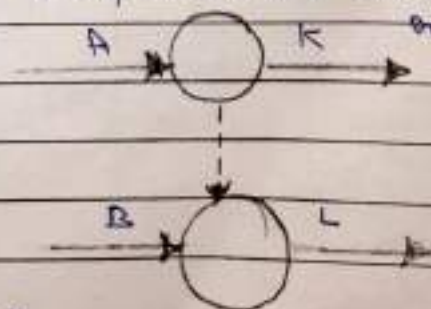
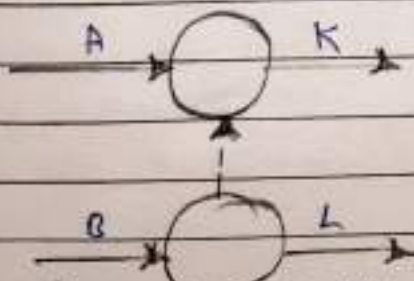
(1)

Activity on Arrow (AOA Diagramming Method)

K, L Depends on A/B



K depends on A but L depends on both A and B

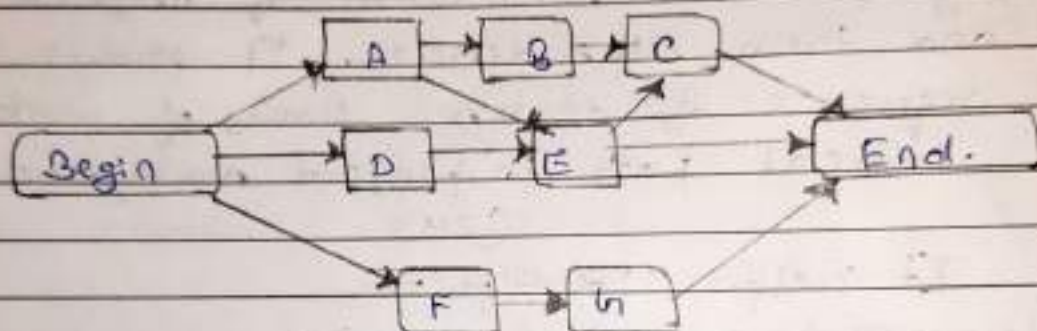


K depends on A, B & L depends on B

- Activity is shown on the Arrow
- Node represents starting & ending events.

(2)

Activity on Node (AON) Diagram - Precedence Diagramming Methods



- Activity is represented on a Node (box)
- Arrows show dependencies.

Others less Common Network Representation

(3) Event on Node (EON) → Shows events but rarely used.

(4) Circle & Arrow Diagram - Traditional PERT charts.